Masked Is Not Harmless: Certified Gate-Level Fault Vulnerability Analysis with LLM Agents That Propose and Engines That Decide

Ali Roshani, Mohsen Ansari, and Mohammad T. Manzuri Shalmani

***Abstract*—Gate-level fault injection labels every fault as silent data corruption (SDC), detected unrecoverable error (DUE), or masked. The masked label mixes faults that are harmless with faults that the workload never exercised, and no current tool says how large the second group is. We present GateScan, a framework that reports vulnerability as a certified bracket instead of a point estimate. The lower end counts faults with a replay-confirmed detection, the upper end removes faults proven harmless by structural or formal arguments, and every sampled quantity carries an interval with exact coverage. Large language model (LLM) agents narrow the bracket by writing workloads and per-fault trigger sequences, but they only propose: a deterministic core computes every verdict, so a wrong agent can widen the bracket and can never make it false. An eleven-class taxonomy refines SDC/DUE/masked by the evidence behind each verdict. On the three ShadowFI benchmarks, 13% to 55% of the faults that the three-class view reports as masked are vulnerabilities proven by replayed witnesses, and the agents raise the certified lower bound of the tensor core unit from 72.5% to 87.5%. On twenty ITC'99 circuits of up to 105k cells, the bracket shows where random workloads fail; we also report the designs on which the agents did not help.**

***Index Terms*—Fault injection, vulnerability analysis, stuck-at faults, soft errors, formal verification, statistical fault injection, large language models, LLM agents, functional safety.**

I. Introduction

WHEN a fault-injection campaign reports that a fault is *masked*, what do we actually know? Safety standards such as ISO 26262 [1] and architectural vulnerability factor (AVF) analyses [2] turn such labels into diagnostic-coverage figures and failure rates, so the answer matters. Since the FARM framework [3], a campaign has been defined by its faults, its activation (the workload and the injection time), its readouts, and its measures. Modern open-source tools such as ShadowFI [4] inject millions of gate-level faults through netlist saboteurs and classify each outcome as silent data corruption (SDC), detected unrecoverable error (DUE), or masked.

The masked class hides two very different situations. A fault may be harmless: no input sequence can ever make it visible. Or the workload may simply never have activated it, or never propagated its effect to an output. The first fault is irrelevant for safety; the second is a vulnerability that the campaign missed. Every tool we surveyed takes the workload from the user, from a fixed benchmark, or from random vectors, and none bounds the share of vulnerable faults among those it calls masked (Section II). A single SDC percentage therefore mixes a property of the circuit with a property of the testbench.

This paper argues that gate-level vulnerability should be reported as a *bracket* whose two ends are backed by different kinds of evidence. The lower end counts faults for which a concrete input sequence, replayed in two simulators, produces an output error. The upper end removes faults proven harmless, either structurally or by a formal proof. Everything in between is *undecided* and is said to be so. The bracket is always correct; its width measures how much the evidence still lacks.

Narrowing the bracket needs better workloads, and writing workloads for an unfamiliar design is exactly the expert task that large language models (LLMs) have begun to automate for RTL code and testbenches [5]–[7]. An LLM, however, is not a sound oracle. Our design principle is therefore a strict division of labor: *the LLM proposes, the deterministic core decides*. Agents write workloads, per-fault trigger sequences, and application-level tolerances as data; a deterministic core turns this data into verdicts, and every number in a report is a function of the netlist, the workloads, the seeds, and solver answers only. A wrong agent can make the bracket wider; it can never make it false.

We implement this principle in GateScan (Fig. 1), an open framework built on Yosys [8], Verilator [9], Icarus Verilog [10], and ABC [11], and we use Claude Opus 5.5 as the LLM in all experiments. The contributions are as follows.

1) A *certified vulnerability bracket* with exact, design-aware statistics (Clopper–Pearson for independent pairs, hypergeometric for samples without replacement, Hoeffding–Serfling for clustered samples) and an *escape certificate* that bounds the share of vulnerable faults among the undecided ones (Sections III and V).
2) An *eleven-class taxonomy* that keeps SDC, DUE, and masked but splits them by evidence: escaped vulnerabilities, delayed SDC, latent, healed, benign latent, proven ineffective, and unknown (Section III).
3) An *agent layer* of five LLM roles, including a per-fault *trigger agent* that receives an activation/propagation diagnosis of each undecided fault, with a soundness argument that no agent output can change a verdict (Sections IV and V).
4) An evaluation on the three ShadowFI benchmarks and twenty ITC'99 circuits [12] that shows what the three-class view hides, how far the agents close the gap, and

Manuscript received October 2026.

The authors are with the Department of Computer Engineering, Sharif University of Technology, Tehran, Iran.

The large language model used in all experiments is Claude Opus 5.5 (Anthropic); every prompt and response is archived with its hash.

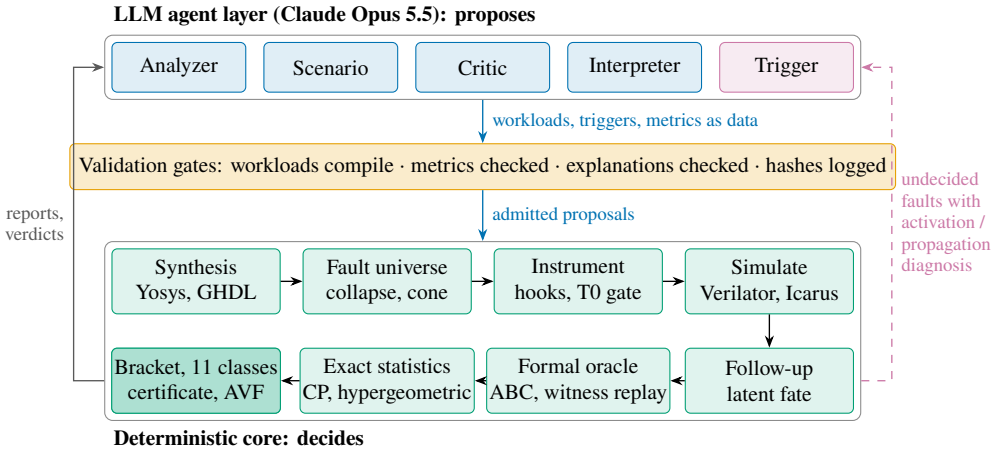


Fig. 1. GateScan architecture. LLM agents (top) receive reports and per-fault diagnoses and return proposals as data. Validation gates admit only well-formed proposals. The deterministic core (bottom) computes every verdict, class, bound, and interval. The trigger loop (dashed) repeats until the undecided set stops shrinking.

where they fail (Section VI).

The rest of the paper follows the question we started with. Section II shows that no prior tool answers it. Section III defines what an answer looks like. Section IV describes the machinery that produces it, and Section V shows why the machinery can be trusted even though an LLM is part of it. Section VI measures the answer on real designs, and Section VII states its limits.

II. Background and Related Work

A. Fault Models, Test, and Fault Injection

Gate-level analysis inherits its fault models from manufacturing test: single stuck-at faults on stems and branches, and transient faults (single-event transients and upsets) for radiation effects [13]–[15]. Test generation answers a related question to ours: whether *some* input sequence detects a fault. Combinational ATPG is complete [16], [17], sequential ATPG is hard in general [18], [19] and is solved in practice by deterministic and genetic search [20], [21], and model checking proves faults untestable through bounded or inductive reasoning [11], [22]–[24]. Fault injection asks a different question: what happens under the *intended* workload. Simulation-based injectors use saboteurs, mutants, or simulator commands [25]–[29], and emulation moves the same idea to FPGAs [30]–[32]. FARM [3] gives the vocabulary used here: faults, activation, readouts, and measures.

B. Statistics of Fault-Injection Campaigns

Powell, Arlat, and co-workers showed that coverage is a property of the joint fault and activation space and gave stratified estimators with confidence limits [33]–[35]. The sample-size formula of Leveugle *et al.* [36] is the most used in practice; it treats the fault location as the only random factor and relies on a normal approximation. Exact binomial intervals [37] are standard elsewhere, but we found no injection tool that reports intervals whose coverage depends on its actual sampling design.

TABLE I
What Recent Tools Report About the Faults They Call Masked

Work	Workload	Classes	CI	Escape	Role of AI
ShadowFI [4]	user	SDC, DUE, masked	no	no	none
RADSAFiE [38]	user	not reported	no	no	none
Geier [39]	benchmark	+ potentially latent	no	no	none
veriDOFS [47]	random	detected, potential	yes	no	none
RIFT [40]	dataset	accuracy drop	seeds	no	fault location
FT-Pilot [41]	manual	error, no error	no	no	hardening
LLM4Cov [7]	LLM	coverage only	no	no	testbench
GateScan	LLM + given	11 classes	exact	yes	workloads, triggers

C. Recent Frameworks and Learning-Based Methods

ShadowFI [4] inserts Yosys-based saboteurs with a scan-configured controller and runs campaigns on HPC simulators or on an FPGA cluster; on four accelerators it reports SDC rates of about 72%–92% for stuck-at faults, with no interval and no latent class, and the workload is a user test application. RADSAFiE [38] inserts LUT-based injection cells into FPGA netlists. Geier *et al.* [39] accelerate RTL injection with checkpoints and a masking check and observe that 46%–97% of experiments end neither in SDC nor in DUE, which quantifies the latent problem. On the learning side, RIFT [40] uses reinforcement learning to choose fault *locations* in neural-network weights, FT-Pilot [41] uses a graph network and an LLM to harden RTL with labels from injection under a hand-written testbench, and graph or neural predictors estimate flip-flop criticality from injection data [42]–[44]. LLM testbench generators optimize functional correctness or code coverage [6], [7], [45], [46], not fault detection.

D. The Gap

Table I summarizes the comparison. Three gaps remain. *G1:* the activation set is always supplied from outside the tool, and learning has been applied to fault locations and to hardening but not to choosing workloads with fault detection as the objective. *G2:* no tool reports intervals matched to its sampling

design or bounds the share of vulnerable faults among those it calls masked. *G3:* latent errors are either merged into masked or reported as unresolved, without a follow-up that decides their fate. The next section defines a report format that closes G2 and G3; Section IV closes G1 without giving the LLM any authority over the result.

III. From a Point Estimate to a Certified Bracket

A. Setting

Let N be the flat gate-level netlist synthesized from the RTL, with state s, inputs u, observed outputs y, and one clock. The fault universe $\mathcal{F}$ holds single stuck-at faults (SA0 and SA1 on stems and primary inputs) and single-event upsets (SEU, one per storage bit); each experiment has exactly one fault. A workload W is an input sequence $u_0, \ldots, u_{H-1}$ whose first r cycles are the reset, with injection cycles $T \subseteq [r, H)$. An experiment (f, τ) simulates N with fault f active from cycle τ (a stuck-at fault stays active; an SEU inverts one bit until its next capture) and compares it with the fault-free run. Its verdict is *detected* if some output differs, *hang* if a liveness output fails to re-assert, *latent* if no output differs but the final state does, *masked* if neither differs, and *untestable-structural* if no structural path leads from the fault site to an output. An engine failure is an *error* and is never counted as masked.

B. The Bracket

Partition the stuck-at faults into three sets. $\mathcal{D}$ holds faults with a definite detection under some workload or a replay-confirmed witness. $\mathcal{R}$ holds faults proven unobservable, structurally or by a formal proof. $\mathcal{U} = \mathcal{F} \setminus (\mathcal{D} \cup \mathcal{R})$ holds every other fault, including every fault that the formal step did not examine. Let V be the share of faults that *some* input sequence from reset can expose.

Definition 1 (Vulnerability bracket)**.** *The bracket of a campaign is* $\big[\, |\mathcal{D}|/|\mathcal{F}|,\ (|\mathcal{D}| + |\mathcal{U}|)/|\mathcal{F}| \,\big]$.

Theorem 2 in Section V shows that V always lies inside the bracket. The lower end is what a classical campaign calls its SDC plus DUE rate, but restricted to definite detections; the upper end is what is left after removing proven harmless faults. When faults are sampled, both ends carry exact intervals for the population. The width $|\mathcal{U}|/|\mathcal{F}|$ is the honest measure of what the evidence lacks, and every component of GateScan exists either to move faults out of $\mathcal{U}$ or to bound how many of them are vulnerable.

For the faults that stay in $\mathcal{U}$, GateScan draws a uniform sample without replacement and asks the formal oracle about each one. A fault with a replayed witness is a proven escape; a solver timeout is counted as an escape. The hypergeometric interval over the sample gives the *escape certificate*: an upper confidence limit on the share of vulnerable faults among the undecided ones. It answers the question of the introduction in numbers.

For SEUs the fault is transient, so the corresponding statement concerns time rather than inputs. GateScan replays the workload after a latent run and reports three nested AVF bounds: within the horizon, within the follow-up, and an upper bound that counts every experiment not shown to be masked (Theorem 5).

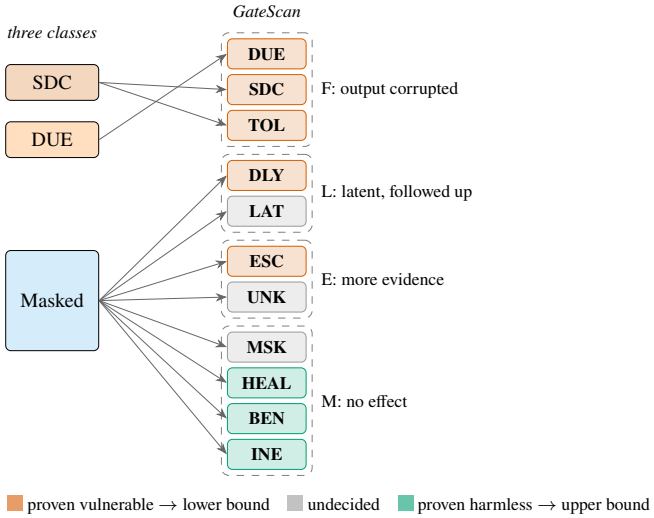


Fig. 2. The three-class view (left) and the eleven GateScan classes (right). Arrows show where each three-class label goes. Red classes are proven vulnerable and define the lower end of the bracket; green classes are proven harmless and define the upper end; grey classes stay undecided and are always counted on the vulnerable side of the upper bound.

C. An Evidence-Based Taxonomy

The bracket needs every experiment to say which kind of evidence stands behind it. Fig. 2 shows the eleven classes, grouped in four families and ordered by severity. Family F holds faults that corrupted an output: DUE (a hang or a declared alarm output), SDC, and TOL (an output differed but every application metric stayed within the tolerance of the interpreter agent, for example two units in the last place). Family L holds latent errors: DLY (a delayed SDC that surfaced during the follow-up) and LAT (still corrupted, fate unknown). Family E holds faults that looked masked but have more evidence: ESC (an escaped vulnerability with a replayed witness from the formal oracle or the trigger agent) and UNK (no verdict). Family M holds MSK (masked under the workload, nothing more is known), HEAL (a transient corruption that returned to the golden state), BEN (a latent error proven unable to reach an output), and INE (proven ineffective for every input).

A fault receives the most severe class over all its workloads. The mapping back to SDC/DUE/masked is fixed, so every GateScan report also contains the three-class view of the same faults, and the two can be compared fault by fault (Section VI). Only F, DLY, and ESC are proven vulnerable, and only HEAL, BEN, and INE are proven harmless; MSK, LAT, and UNK carry no proof either way. The single most important difference from the three-class view is therefore that *masked* becomes four classes with very different meanings: ESC, MSK, BEN or HEAL, and INE.

IV. GateScan: Agents Propose, the Core Decides

Fig. 1 shows the two layers of GateScan. The deterministic core turns a netlist, a fault set, and workloads into verdicts,

bounds, and intervals. The agent layer reads what the core reports and returns proposals as data. A proposal enters the core only through a validation gate, and it can influence only *which* experiments are run, never how they are judged.

A. The Deterministic Core

Synthesis and fault universe. Yosys synthesizes Verilog or VHDL (through GHDL) to a generic-cell netlist with redundancy preservation, so that registers written in the RTL are not merged; an audit compares RTL and netlist storage bits and warns when they differ. The fault universe is collapsed by structural equivalence, and faults with no path to an observed output are marked untestable-structural. For a campaign restricted to a scope (a set of module instances), only the cone of influence of the scope is simulated, which keeps every verdict and divergence cycle unchanged (Theorem 7).

Instrumentation. Each fault receives exactly one hook and each hook exactly one fault, which a bijection gate checks. A linear-time structural argument, and a SAT equivalence check for small designs, show that the instrumented netlist equals the original one when injection is off; every run also replays the golden trace with injection off and compares it bit for bit.

Simulation and follow-up. Verilator executes the campaign in parallel shards; Icarus Verilog, a four-valued simulator, is the reference engine for golden traces, differential checks, and witness replay. With both engines, only definite (0 versus 1) detections count; a detection seen only through an unknown value is reported as UNK. A latent run is continued by replaying the workload without a new reset, which decides whether the error surfaces, heals, or persists.

Formal oracle. Undecided faults are sampled uniformly without replacement and each sampled fault is encoded as a miter between the fault-free and the faulty netlist under free inputs. ABC [11] (`dprove`) or Yosys induction either proves the fault untestable, returns a counterexample, or runs out of time. A counterexample becomes an ordinary workload and counts only after it detects the fault in both simulators. Proofs are trusted; witnesses are not.

Statistics and report. Every share comes with the exact interval of its sampling design (Section V). The report contains the bracket, the escape certificate, the eleven classes and the three-class view, SEU AVF bounds, per-region and per-register breakdowns, and the provenance of every input.

B. The Agent Layer

Five LLM roles share one interface: a prompt built from the core's data, a JSON answer, and a validator. The *analyzer* reads the port list, hierarchy, clocks, resets, and an RTL excerpt, and returns the role of each port (data, control, handshake) and the operating modes of the design. The *scenario agent* writes workloads in a small declarative language (phases, per-port streams of random, constant, or explicit values); each workload must compile against the design. The *critic* checks a workload against the analyzer's facts and the last campaign report and may request a revision. The *interpreter* writes an application metric (for example, outputs decoded as IEEE-754 numbers with a tolerance in units in the last place) and explanations of the observed outcomes; the metric can only move an SDC to TOL, and each explanation is checked against the measured fates. The *trigger agent* is described next.

Every exchange is logged with the model identity, the prompt and response hashes, the temperature, and the token counts. In this paper all roles were played by Claude Opus 5.5 through a file-based exchange: the core writes a prompt, the model writes the response, and the core validates it.

C. The Per-Fault Trigger Loop

Workload agents reason about the design as a whole. Some faults, however, stay undecided because of one specific condition, and a design-level workload rarely produces it. The trigger loop targets them directly. A probe build of the netlist records, for each undecided stuck-at-v fault, the value of its site during the workload. If the site never takes the value $\bar{v}$, the fault was never *activated*; otherwise it was activated but its effect never *propagated* to an output. The trigger agent receives this diagnosis, grouped by module and signal, together with the design facts, and writes workloads that aim at the missing condition: rare operand combinations for activation, or sequences that keep a corrupted value alive until it is read for propagation. The core then runs every undecided fault under the new workloads, and each new detection enters $\mathcal{D}$ only through the same definite, replayed path as any other detection. The loop stops when a round no longer shrinks $\mathcal{U}$ appreciably.

V. Guarantees

An LLM inside a safety analysis is acceptable only if the analysis remains correct when the LLM is wrong. This section states what GateScan guarantees, under which assumptions, and how each assumption is enforced; proofs are in Appendix A.

A. Assumptions

GateScan distinguishes *checked* assumptions, which a gate verifies in every campaign and which stop the campaign when they fail, from *stated* assumptions, which appear in every report. Checked: (A2) the instrumented netlist with injection off equals the original; (A3) hooks and faults are in bijection; (A4) the engines follow the simulation protocol, verified by Verilator–Icarus golden-trace equality, differential verdicts, and two-engine replay of every witness. Stated: (A1) the netlist behaves like the RTL on its ports, supported by the storage-bit audit; (A5) one fault per experiment; (A6) solver proofs of untestability are sound, validated against exhaustive simulation; (A8) the workload is a legal behavior of the environment, while formal witnesses may use inputs that the real environment never produces.

B. Theorems

Theorem 1 (Sound verdicts)**.** *Under A1–A4, a definite detection or hang under W implies that the RTL with the fault produces an output difference under W. Under A2,*

A4, and A5, a masked or healed SEU is masked for every continuation of the inputs, a benign latent error never reaches an observed output, and a fault proven untestable is detected by no workload.

Theorem 2 (Bracket)**.** $|\mathcal{D}|/|\mathcal{F}| \le V \le (|\mathcal{D}| + |\mathcal{U}|)/|\mathcal{F}|$.

Theorem 3 (Independence from the LLM)**.** *Every verdict, class, bound, and interval is a deterministic function of the netlist, the fault set, the workloads, the seeds, and the solver answers. An arbitrarily wrong agent can only lower coverage, which widens the bracket, or loosen a tolerance, which changes only the application-level AVF reported next to the raw bounds.*

Theorem 4 (Monotonicity)**.** *For workload sets* $\mathcal{W} \subseteq \mathcal{W}'$, $\mathcal{D}(\mathcal{W}) \subseteq \mathcal{D}(\mathcal{W}')$. *Because every agent flow keeps the given or random workload, agents can never lower the lower end of the bracket.*

Theorem 5 (SEU AVF bounds)**.** *Let* AVF_∞ *be the share of SEU experiments whose error ever reaches an output when the workload continues. Then the share detected within the horizon is at most the share detected within the follow-up, which is at most* AVF_∞*, which is at most the share not shown to be masked.*

Theorem 6 (Exact coverage)**.** *Each reported interval covers the true share with probability at least 0.95 for every value of that share: exact for a census; Clopper–Pearson [37] for independent fault–cycle pairs; the inverted hypergeometric test for faults sampled without replacement; Hoeffding–Serfling [48], [49] for samples without replacement with several cycles per fault. The escape certificate is a hypergeometric interval over the residue, with every unknown counted as an escape.*

Theorem 7 (Exact cone reduction)**.** *Simulating only the cone of influence of a scope gives, for every fault of the scope and every input sequence, the same verdict and the same first divergence cycle as the full netlist.*

Theorem 3 is the formal content of the slogan in the title. The agents can make the analysis *weaker*, never *wrong*; how much stronger they make it is an empirical question, answered in Section VI.

C. Validation of the Core

The theorems hold if the implementation does what the proofs assume, so the core was tested against independent references. On a catalog of 15 circuits, all 165 campaign gates passed, and 1,834 experiments run in both engines gave no verdict disagreement (104 detections depended on unknown values and were classified UNK). On four combinational designs, the formal oracle agreed with exhaustive simulation on every fault. The follow-up equals a plain run on the extended workload on all 22 design and workload pairs. The hypergeometric interval reaches coverage of at least 0.95 for all 41 population counts of an exhaustive check, and the Clopper–Pearson intervals reached 0.96–1.0 in calibration studies where the Wald interval reached only 0.90–0.947. Mutation testing of the core killed 22 of 23 mutants; the survivor is semantically equivalent.

VI. Evaluation

We ask three questions. *RQ1:* what does the three-class view hide on designs where published results exist? *RQ2:* do LLM agents narrow the bracket, and when do they fail? *RQ3:* how does the approach scale, and what does it cost?

A. Setup

All campaigns ran on one workstation with 16 hardware threads and a Linux virtual machine limited to 11 GB of memory, with Yosys 0.33, Verilator 5.020, Icarus Verilog 12.0, and ABC from Yosys. The ShadowFI benchmarks [4] are the two special function units (SFUv1, SFUv2) and the 4×4 tensor core unit (TCU) from its public repository, each with its own test application (140,062 operations for SFUv1, 4,096 matrix products for TCU, and 880,083 vectors for SFUv2). The fourth benchmark, the stereo vision core, could not be compiled: Verilator exceeded the 11 GB limit. Each campaign targets a scope: the squaring unit and exception logic of SFUv1, one floating-point adder of the TCU dot-product unit, and the 24 arithmetic submodules of SFUv2. We sampled 3,000 stuck-at faults without replacement in SFUv1 and TCU and simulated all 2,720 of SFUv2. The formal oracle examined a uniform sample of 60 undecided faults per design with a 60 s budget per fault. SEU campaigns used 2,450 independent fault–cycle pairs. The ITC'99 circuits [12] used a uniform random workload of 2,000 cycles, a census of the faults, except for b17, b18, b19, and b22, where 2,450 faults were sampled without replacement; the formal oracle examined up to 100 undecided faults per circuit.

B. RQ1: What the Three-Class View Hides

Table II and Fig. 3 compare the two views of the same faults. The SDC rates are close to the published ones (94.3% versus 92%, 72.3% versus 72%, 86.4% versus 80%). They are not the same estimand, because ShadowFI reports whole-design rates read from bar charts and we report scoped rates with exact intervals; we therefore use them only as a sanity check.

The difference lies in the masked class. Of the 171, 831, and 369 faults that the three-class view calls masked, 70 (41%), 457 (55%), and 48 (13%) are ESC: a witness, replayed in both simulators, makes each of them corrupt an output. On SFUv1 and SFUv2 the witnesses come from the formal oracle alone, from a sample of 60 representatives; on TCU, six come from the oracle and the rest from the trigger agent (RQ2). Only seven faults, all in SFUv1, were proven ineffective. The rest of the masked class stays MSK, which a report must not read as harmless. The three-class view also hides six latent errors and 28 detections that were visible only through unknown values in TCU. For the SFUv1 outputs, the interpreter agent's tolerance of two units in the last place moves 569 SDC faults to TOL, so the application-level AVF is about 75% while the raw failure rate is 94%; both are reported.

TABLE II
GATESCAN ON THE SHADOWFI BENCHMARKS (STUCK-AT FAULTS). SHADOWFI VALUES ARE WHOLE-DESIGN RATES READ FROM THE PUBLISHED CHARTS

			SDC (%)		Bracket (%)		Faults		s/fault	
Design	Cells	Faults	GateScan, 95% CI	ShadowFI	given	with agents	masked	ESC	GateScan	ShadowFI
SFUv1	22,344	3,000 of 3,976	94.3 [93.9, 94.7]	92	[96.6, 99.8]	[98.8, 99.5]	171	70	0.78	4.7
TCU	103,280	3,000 of 3,524	72.3 [71.7, 72.9]	72	[72.5, 100]	[87.5, 100]	831	457	25.6	1.4
SFUv2	44,782	2,720 (census)	86.4 (exact)	80	[88.2, 100]	[94.7, 100]	369	48	42.6	16.0

Bracket ends are sample shares; the 95% population lower limit of the TCU bracket with agents is 87.1%. s/fault: single-core seconds per fault including follow-up and formal steps for GateScan; Verilator workflow with 3k saboteurs for ShadowFI [4].

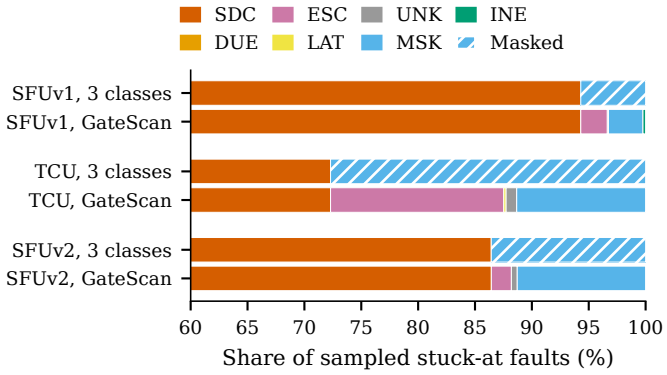


Fig. 3. The same faults in the three-class view (hatched bars) and in the GateScan classes (solid bars). A large part of what the three-class view calls masked is ESC, a vulnerability proven by a replayed witness.

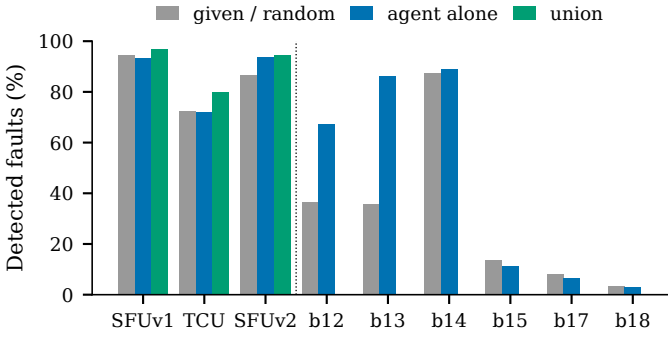


Fig. 5. Detected faults under the given (ShadowFI) or random (ITC'99) workload, under the agents' workload alone, and under their union. Agent workloads help on protocol-gated designs (b12, b13, SFUv2) and fail on the 80386 subsets (b15–b18).

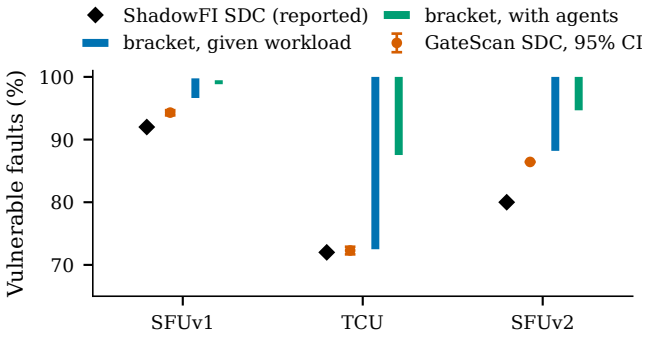


Fig. 4. Point estimates versus certified brackets on the ShadowFI benchmarks. The SDC rate is a property of the workload; the bracket bounds the property of the circuit, and the agents shrink it from below.

Fig. 4 places the point estimates next to the brackets. With the given workloads, the bracket is narrow for SFUv1 but open up to 100% for TCU and SFUv2: there, no amount of sampling makes the SDC rate an estimate of the vulnerability, because the missing faults were never exercised. The SEU AVF of the TCU scope is 74.7% [72.9, 76.4]; it covers the flip-flops of the scope only and is not comparable with ShadowFI's whole-design SEU rate of about 18%.

C. RQ2: Do the Agents Narrow the Bracket?

Design-level workloads. Fig. 5 compares the given workload with the agents' workloads. On the ShadowFI designs, the agent workloads alone detect about as many faults as the test applications on SFUv1 and TCU (93.2% versus 94.3%, 71.9% versus 72.3%) and more on SFUv2 (93.8% versus 86.4%), while being 10 to 80 times shorter (14k and 11k cycles against 140k and 880k vectors). The two workloads detect different faults: 73, 232, and 224 faults are detected only by the agents, so the union raises the lower end of the bracket to 96.7%, 80.0%, and 94.7%, and the SFUv1 bracket closes to [98.8, 99.5].

On ITC'99, the scenario agent diagnosed why random inputs fail on the larger circuits: random control pins break handshakes and restart state machines. On b12 (a guessing game) and b13 (a sensor interface), keeping the start pulse and the converter handshake in their legal order raised detection from 36.6% to 67.1% and from 35.7% to 86.0%, and the b13 bracket narrowed to [87.4, 93.6]. On b14 a ten times longer instruction stream gave a small gain (87.3% to 88.9%). On b15, b17, and b18 (subsets of an 80386 processor), the agent's hypothesis that random bus-control signals starve the processor was wrong: detection fell slightly (13.7% to 11.2%, 8.0% to 6.7%, 3.2% to 3.0%). By Theorem 4 this failure costs time but no correctness: the union with the random workload keeps the original lower bound.

Per-fault triggers. TCU is the hardest case: after the given and design-level workloads, 825 of the 3,000 sampled faults stayed undecided, and the formal oracle decided only 6 of its 60 sampled faults within the budget. Fig. 6 shows the trigger loop. In round 1 the diagnosis found 273 faults never activated and 552 activated but not propagated; the agent wrote workloads with operands isolated per lane and special floating-

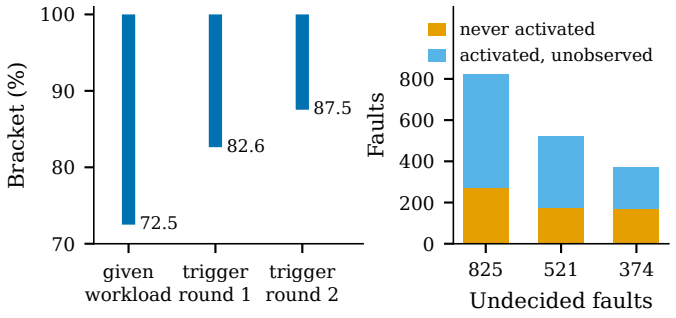


Fig. 6. The trigger loop on TCU. Left: the lower end of the bracket after each round. Right: the diagnosis of the undecided faults before each round, and of the 374 faults left after round 2.

point values, which detected 304 more faults. In round 2, on the 521 remaining faults, it wrote operands with close exponents, which exercise alignment and cancellation in the adder, and corner cases, which detected 147 more. The lower end of the bracket rose from 72.5% to 82.6% and to 87.5% (population lower limit 87.1%). Of the 374 faults that remain, 171 were never activated, and 137 of them sit on signals that are constant 0 under every workload we tried; they are likely untestable, but the oracle could not prove it within the budget.

D. RQ3: Scale and Cost

Fig. 7 shows the brackets of twenty ITC'99 circuits, from 39 to 104,873 cells and from 82 to 209,838 faults; Table III in Appendix B lists all numbers. On small circuits the bracket is closed or narrow, and the formal oracle does much of the work: on b09 it turned 81.5% detection into an exact 98.0%, and on b02, b03, b06, and b10 the vulnerability is known exactly. On the large circuits, random workloads detect 1.5%–13.7% of the faults while the brackets reach up to 97%–100%, and the oracle returned no answer for any of the 100 sampled faults of b18 and b19 within the budget. A point estimate of 3% for b18 would be read as "the circuit is robust"; the bracket [3.2, 99.6] says that the campaign knows almost nothing.

GateScan is slower per fault than ShadowFI on two of the three designs (Table II): 25.6 s against 1.4 s on TCU and 42.6 s against 16 s on SFUv2, on one core, while it is faster on SFUv1 (0.78 s against 4.7 s). The comparison is not like for like: our SFUv2 workload is almost nine times longer than ShadowFI's 100,000 operations, and our per-fault time includes the two-engine checks, the follow-up of latent runs, and the formal step. With 16 threads the wall-clock time per fault is 0.11, 3.6, and 5.4 s; campaigns are independent per fault, so the same shards could run on a cluster or on the FPGA back end of ShadowFI. The largest campaign, b19 with a sample of 2,450 faults, took 3.4 h.

VII. Discussion and Limitations

What the agents contribute. The agents never decide anything, and their value is measured only by how much they shrink $\mathcal{U}$. That value was large where a short, legal, well-ordered stimulus is what the design needs (b12, b13, SFUv2, and the per-fault triggers on TCU), and zero where the agent's model of the design was wrong (b15–b18). The second case is the reason for the architecture: a plausible but wrong hypothesis produced a slightly weaker workload, and nothing in the report became false. A per-fault trigger round on the 80386 subsets, as on TCU, is the natural next step.

Formal scaling. The oracle decides most sampled faults on small and medium designs but almost none on b18, b19, and the TCU adder within 60 s. Longer budgets, cone-restricted miters, and property-directed reachability [24] would help; until then, the escape certificate on large designs is conservative, because every timeout is counted as an escape.

Validity. The guarantees rest on stated assumptions: synthesis fidelity, sound solver proofs, a single fault per experiment, and single-clock designs. Formal witnesses may use inputs that the real environment never produces, so an ESC fault is vulnerable *structurally*, not necessarily in operation; reports keep the two apart. Our netlists use generic cells, and stuck-at coverage over-estimates defect coverage [47]. The ShadowFI comparison uses scoped campaigns, and one benchmark could not be compiled within our memory limit.

The LLM. All agent roles were played by one model, Claude Opus 5.5, in an interactive file exchange. Theorem 3 makes the correctness of the results independent of this choice, but the size of the gains is not: another model, or the same model at a different temperature, may write better or worse workloads. Every prompt, response, and resulting workload is archived with its hash so that the gains can be reproduced or challenged.

VIII. Conclusion

A fault that a campaign calls masked may be harmless, or it may be a vulnerability the workload never reached. GateScan replaces the single SDC rate with a certified bracket that keeps these cases apart, an eleven-class taxonomy that names the evidence behind each verdict, and exact intervals for every sampled share. LLM agents write workloads and per-fault triggers that shrink the bracket, while a deterministic core keeps every number independent of them. On the ShadowFI benchmarks, 13% to 55% of the faults that the three-class view calls masked are proven vulnerabilities, and the trigger agent raised the certified lower bound of the tensor core adder from 72.5% to 87.5%. On ITC'99, the bracket shows which results can be trusted and which cannot. The open problems are the formal oracle on large designs and agents for processor-like designs, where the first hypothesis of our agent was wrong.

References

[1] *Road Vehicles—Functional Safety*, ISO 26262:2018, Dec. 2018.
[2] S. S. Mukherjee, C. Weaver, J. Emer, S. K. Reinhardt, and T. Austin, "A systematic methodology to compute the architectural vulnerability factors for a high-performance microprocessor," in *Proc. 36th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO-36)*, San Diego, CA, USA, Dec. 2003, pp. 29–40, doi: 10.1109/MICRO.2003.1253181.
[3] J. Arlat *et al.*, "Fault injection for dependability validation: A methodology and some applications," *IEEE Trans. Softw. Eng.*, vol. 16, no. 2, pp. 166–182, Feb. 1990, doi: 10.1109/32.44380.
[4] J.-D. Guerrero-Balaguera *et al.*, "SHADOWFI: An open-source framework for fault evaluation of complex IC designs using hyperscale computing," *IEEE Access*, vol. 13, pp. 211 382–211 406, 2025, doi: 10.1109/ACCESS.2025.3641762.



Fig. 7. Brackets on twenty ITC'99 circuits, ordered by size (cells below each name). Short bars are decided circuits; long bars show where a random workload leaves most faults undecided. Triangles mark the lower end reached by the agent workloads on b12–b18.

[5] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Invited paper: VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. (ICCAD)*, San Francisco, CA, USA, Oct. 2023, pp. 1–8, doi: 10.1109/ICCAD57390.2023.10323812.

[6] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design," in *Proc. ACM/IEEE Int. Symp. Mach. Learn. CAD (MLCAD)*, Salt Lake City, UT, USA, Sep. 2024, pp. 1–10, doi: 10.1145/3670474.3685956.

[7] H. Zhang, Z. Yu, C.-T. Ho, H. Ren, B. Khailany, and J. Zhao, "LLM4Cov: Execution-aware agentic learning for high-coverage testbench generation," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, 2026, arXiv:2602.16953. [Online]. Available: https://arxiv.org/abs/2602.16953

[8] C. Wolf. Yosys open SYnthesis suite. YosysHQ. Accessed: Oct. 21, 2026. [Online]. Available: https://yosyshq.net/yosys/

[9] W. Snyder *et al.* Verilator. Veripool. Accessed: Oct. 21, 2026. [Online]. Available: https://www.veripool.org/verilator/

[10] S. Williams. Icarus Verilog. Accessed: Oct. 21, 2026. [Online]. Available: https://steveicarus.github.io/iverilog/

[11] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. Int. Conf. Comput. Aided Verification (CAV)*, ser. Lecture Notes in Computer Science, vol. 6174. Berlin, Germany: Springer, 2010, pp. 24–40, doi: 10.1007/978-3-642-14295-6_5.

[12] F. Corno, M. Sonza Reorda, and G. Squillero, "RT-level ITC'99 benchmarks and first ATPG results," *IEEE Design & Test of Computers*, vol. 17, no. 3, pp. 44–53, 2000.

[13] M. Abramovici, M. A. Breuer, and A. D. Friedman, *Digital Systems Testing and Testable Design*. New York, NY, USA: IEEE Press, 1990, doi: 10.1109/9780470544389.

[14] M. L. Bushnell and V. D. Agrawal, *Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits*, ser. Frontiers in Electronic Testing. Boston, MA, USA: Kluwer Academic, 2000, doi: 10.1007/b117406.

[15] R. C. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," *IEEE Trans. Device Mater. Rel.*, vol. 5, no. 3, pp. 305–316, Sep. 2005, doi: 10.1109/TDMR.2005.853449.

[16] J. P. Roth, "Diagnosis of automata failures: A calculus and a method," *IBM J. Res. Develop.*, vol. 10, no. 4, pp. 278–291, Jul. 1966, doi: 10.1147/rd.104.0278.

[17] P. Goel, "An implicit enumeration algorithm to generate tests for combinational logic circuits," *IEEE Trans. Comput.*, vol. C-30, no. 3, pp. 215–222, Mar. 1981, doi: 10.1109/TC.1981.1675757.

[18] O. H. Ibarra and S. K. Sahni, "Polynomially complete fault detection problems," *IEEE Trans. Comput.*, vol. C-24, no. 3, pp. 242–249, Mar. 1975, doi: 10.1109/T-C.1975.224205.

[19] H. Fujiwara and S. Toida, "The complexity of fault detection problems for combinational logic circuits," *IEEE Trans. Comput.*, vol. C-31, no. 6, pp. 555–560, Jun. 1982, doi: 10.1109/TC.1982.1676041.

[20] T. M. Niermann and J. H. Patel, "HITEC: A test generation package for sequential circuits," in *Proc. Eur. Conf. Design Autom. (EDAC)*, Amsterdam, The Netherlands, Feb. 1991, pp. 214–218, doi: 10.1109/EDAC.1991.206393.

[21] F. Corno, P. Prinetto, M. Rebaudengo, and M. Sonza Reorda, "GATTO: A genetic algorithm for automatic test pattern generation for large synchronous sequential circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 15, no. 8, pp. 991–1000, Aug. 1996, doi: 10.1109/43.511578.

[22] A. Biere, A. Cimatti, E. Clarke, and Y. Zhu, "Symbolic model checking without BDDs," in *Proc. Int. Conf. Tools Algorithms Construct. Anal. Syst. (TACAS)*, ser. Lecture Notes in Computer Science, vol. 1579. Berlin, Germany: Springer, 1999, pp. 193–207, doi: 10.1007/3-540-49059-0_14.

[23] M. Sheeran, S. Singh, and G. Stålmarck, "Checking safety properties using induction and a SAT-solver," in *Proc. Formal Methods Comput.-Aided Des. (FMCAD)*, ser. Lecture Notes in Computer Science, vol. 1954. Berlin, Germany: Springer, 2000, pp. 127–144, doi: 10.1007/3-540-40922-X_8.

[24] N. Eén, A. Mishchenko, and R. Brayton, "Efficient implementation of property directed reachability," in *Proc. Formal Methods Comput.-Aided Des. (FMCAD)*, Austin, TX, USA, Oct. 2011, pp. 125–134.

[25] E. Jenn, J. Arlat, M. Rimén, J. Ohlsson, and J. Karlsson, "Fault injection into VHDL models: The MEFISTO tool," in *Proc. 24th Int. Symp. Fault-Tolerant Comput. (FTCS-24)*, Austin, TX, USA, Jun. 1994, pp. 66–75, doi: 10.1109/FTCS.1994.315656.

[26] J. C. Baraza, J. Gracia, D. Gil, and P. J. Gil, "A prototype of a VHDL-based fault injection tool: Description and application," *J. Syst. Archit.*, vol. 47, no. 10, pp. 847–867, Apr. 2002, doi: 10.1016/S1383-7621(01)00036-4.

[27] J. C. Baraza, J. Gracia, S. Blanc, D. Gil, and P. J. Gil, "Enhancement of fault injection techniques based on the modification of VHDL code," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 16, no. 6, pp. 693–706, Jun. 2008, doi: 10.1109/TVLSI.2008.2000254.

[28] A. Benso and P. Prinetto, Eds., *Fault Injection Techniques and Tools for Embedded Systems Reliability Evaluation*, ser. Frontiers in Electronic Testing, vol. 23. Boston, MA, USA: Kluwer Academic, 2003, doi: 10.1007/b105828.

[29] H. Ziade, R. Ayoubi, and R. Velazco, "A survey on fault injection techniques," *Int. Arab J. Inf. Technol.*, vol. 1, no. 2, pp. 171–186, Jul. 2004.

[30] P. Civera, L. Macchiarulo, M. Rebaudengo, M. Sonza Reorda, and M. Violante, "Exploiting circuit emulation for fast hardness evaluation," *IEEE Trans. Nucl. Sci.*, vol. 48, no. 6, pp. 2210–2216, Dec. 2001, doi: 10.1109/23.983197.

[31] C. López-Ongil, M. García-Valderas, M. Portela-García, and L. Entrena, "Autonomous fault emulation: A new FPGA-based acceleration system for hardness evaluation," *IEEE Trans. Nucl. Sci.*, vol. 54, no. 1, pp. 252–261, Feb. 2007, doi: 10.1109/TNS.2006.889115.

[32] L. Entrena, M. García-Valderas, R. Fernández-Cardenal, A. Lindoso, M. Portela, and C. López-Ongil, "Soft error sensitivity evaluation of microprocessors by multilevel emulation-based fault injection," *IEEE Trans. Comput.*, vol. 61, no. 3, pp. 313–322, Mar. 2012, doi: 10.1109/TC.2010.262.

[33] D. Powell, E. Martins, J. Arlat, and Y. Crouzet, "Estimators for fault tolerance coverage evaluation," *IEEE Trans. Comput.*, vol. 44, no. 2, pp. 261–274, Feb. 1995, doi: 10.1109/12.364537.

[34] D. Powell, M. Cukier, and J. Arlat, "On stratified sampling for high coverage estimations," in *Proc. 2nd Eur. Dependable Comput. Conf. (EDCC-2)*, ser. Lecture Notes in Computer Science, A. Hlawiczka, J. G. Silva, and L. Simoncini, Eds., vol. 1150. Berlin, Germany: Springer, Oct. 1996, pp. 35–54, doi: 10.1007/3-540-61772-8_29.

[35] M. Cukier, D. Powell, and J. Arlat, "Coverage estimation methods for

stratified fault-injection," *IEEE Trans. Comput.*, vol. 48, no. 7, pp. 707–723, Jul. 1999, doi: 10.1109/12.780878.

[36] R. Leveugle, A. Calvez, P. Maistri, and P. Vanhauwaert, "Statistical fault injection: Quantified error and confidence," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Nice, France, Apr. 2009, pp. 502–506, doi: 10.1109/DATE.2009.5090716.

[37] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi: 10.1093/biomet/26.4.404.

[38] M. Monopoli, M. Biondi, P. Nannipieri, S. Moranti, and L. Fanucci, "RADSAFiE: A netlist-level fault injection user interface application for FPGA-based digital systems," *IEEE Access*, vol. 13, pp. 28 809–28 823, 2025, doi: 10.1109/ACCESS.2025.3539932.

[39] J. Geier, D. Mueller-Gritschneder, and U. Schlichtmann, "Techniques and tools for fast fault injection simulations of RISC-V processors at RTL," presented at the RISC-V Summit Europe, Munich, Germany, Jun. 2024. [Online]. Available: https://mediatum.ub.tum.de/doc/1748212/ase7j1kkihpymrdbej3vytrvk.geier_rvsummiteu24_ea.pdf

[40] K. Khalil, M. M. Khaliq, and K. A. Hoque, "RIFT: A scalable methodology for LLM accelerator fault assessment using reinforcement learning," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Apr. 2026, pp. 1–7, doi: 10.23919/DATE69613.2026.11539080.

[41] W. Liu *et al.*, "FT-Pilot: Automated fault-tolerant RTL rewriting via vulnerability-guided LLMs," arXiv:2605.28169, May 2026. [Online]. Available: https://arxiv.org/abs/2605.28169

[42] A. Balakrishnan, T. Lange, M. Glorieux, D. Alexandrescu, and M. Jenihhin, "Modeling gate-level abstraction hierarchy using graph convolutional neural networks to predict functional de-rating factors," in *Proc. NASA/ESA Conf. Adaptive Hardw. Syst. (AHS)*, Colchester, U.K., Jul. 2019, pp. 72–78, doi: 10.1109/AHS.2019.00007.

[43] L. Lu, J. Chen, M. Ulbricht, and M. Krstic, "Toward critical flip-flop identification for soft-error tolerance with graph neural networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 4, pp. 1135–1148, Apr. 2024, doi: 10.1109/TCAD.2023.3331968.

[44] T. Lange, A. Balakrishnan, M. Glorieux, D. Alexandrescu, and L. Sterpone, "On the estimation of complex circuits functional failure rate by machine learning techniques," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.—Suppl. Vol. (DSN-S)*, Portland, OR, USA, Jun. 2019, pp. 35–41, doi: 10.1109/DSN-S.2019.00021.

[45] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "CorrectBench: Automatic testbench generation with functional self-correction using LLMs for HDL design," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Lyon, France, Mar. 2025, pp. 1–7, doi: 10.23919/DATE64628.2025.10992873.

[46] C.-C. Meng, Y.-R. Lu, G.-Y. Lin, T. T. Yeh, K.-C. Wu, and I.-C. Wu, "HAVEN: Hybrid automated verification ENgine for UVM testbench synthesis with LLMs," arXiv:2604.27643, Apr. 2026. [Online]. Available: https://arxiv.org/abs/2604.27643

[47] M. B. Santos and J. P. Teixeira, "Defect-oriented mixed-level fault simulation of digital systems-on-a-chip using HDL," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Munich, Germany, Mar. 1999, pp. 549–553, doi: 10.1109/DATE.1999.761181.

[48] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 13–30, 1963.

[49] R. J. Serfling, "Probability inequalities for the sum in sampling without replacement," *The Annals of Statistics*, vol. 2, no. 1, pp. 39–48, 1974.

APPENDIX A
PROOFS

Theorem 1. By A2 and the bit-exact golden replay, the instrumented netlist with injection off reproduces N. By A4, the faulty run is N with the fault applied as the protocol prescribes. A reported cycle k with $y_k^f \neq y_k$ is therefore a difference in N, and A1 transfers it to the RTL. With both engines, a definite detection is a 0-versus-1 difference in the four-valued simulator, so it holds for every resolution of uninitialized values. A witness is itself a workload and is replayed in both engines. For the negative part, an SEU leaves no trace except in the state; the compared state includes the flip register of the upset, so equal states at a cycle c and equal inputs from c on imply equal states and outputs for all later cycles by determinism. For benign latent errors, let C be the set of state bits in the transitive fan-in of the observed outputs, closed through storage. A bit outside C drives no bit inside C, so the projection of the state on C evolves identically in both runs, and the outputs depend only on the inputs and on C; benignity is claimed only when every differing bit is listed and lies outside C. An untestable-structural fault is the same argument with an empty difference inside C. A fault proven untestable has a miter of the fault-free and faulty netlists under free inputs whose outputs never differ from the initial states of the proof, and every workload is one such input sequence.

Theorem 2. Theorem 1 gives $\mathcal{D} \subseteq \mathcal{T}$, where $\mathcal{T}$ is the set of faults that some input sequence exposes, and $\mathcal{R} \cap \mathcal{T} = \emptyset$. Since $\mathcal{F} = \mathcal{D} \sqcup \mathcal{R} \sqcup \mathcal{U}$, we have $|\mathcal{D}| \leq |\mathcal{T}| \leq |\mathcal{D}| + |\mathcal{U}|$.

Theorem 3. Agent workloads must compile against the design, so each is an input sequence in the domain of the verdict function. The metric specification is validated and applied by deterministic code to measured values; it can only relabel SDC as TOL and never touches hangs, latent, or masked outcomes, and the raw bounds are always reported. Explanations are text, never enter a number, and are checked against the measured fates. Hence every number is a function of the netlist, faults, workloads, seeds, and solver answers.

Theorem 4. $\mathcal{D}(\mathcal{W}) = \bigcup_{W \in \mathcal{W}} \mathcal{D}(W)$ is a union of sets.

Theorem 5. The first two inequalities hold because the event set grows with time. The only outcomes excluded from the upper bound are masked, healed, benign latent, and untestable-structural, which by Theorem 1 never reach an output.

Theorem 6. For a census the share is exact. With independent pairs, each experiment draws a fault and a cycle uniformly and independently, and the outcome is a deterministic function of the pair, so the count is binomial and the Clopper–Pearson interval has coverage at least 0.95 by construction. With a fault sample without replacement and one cycle, the count is hypergeometric; the family has a monotone likelihood ratio in the number of vulnerable faults, so inverting two one-sided tests at level 0.025 gives coverage at least 0.95 for every value, which we also verified exhaustively for a population of 40 and a sample of 12. With several cycles per fault, the per-fault share lies in $[0, 1]$ and the Hoeffding–Serfling inequality $P(|\bar{y} - \mu| \geq \varepsilon) \leq 2\exp\big(-2n\varepsilon^2/(1 - (n-1)/N)\big)$ gives the interval. Within a subgroup, conditional on its sample size, the sample remains uniform over the subgroup, so coverage holds conditionally and hence unconditionally. The escape certificate is the hypergeometric case with the residue as population, and counting unknowns as escapes makes the upper limit conservative.

Theorem 7. An output outside the cone has no structural path from any faulty site, so it equals its golden trace. Every output inside the cone is a function of the inputs and of the cells in its fan-in, all of which are kept, so by induction over cycles the kept state and outputs are identical in both netlists. State outside the cone cannot reach a kept output, so a latent error there is benign.

APPENDIX B
ITC'99 RESULTS AND REPRODUCIBILITY

Table III lists the ITC'99 results behind Fig. 7. Circuits b08 and b30 could not be synthesized by the VHDL front end, and b16 is not part of the distribution we used.

TABLE III
ITC'99 STUCK-AT BRACKETS AND SEU AVF (95% INTERVALS)

	Cells	FFs	Faults	Det.	Bracket	Agent	T/U/?	SEU AVF
b02	39	4	82	98.8	[98.8, 98.8]	–	0/1/0	100.0 [99.8, 100.0]
b01	59	5	124	99.2	[99.2, 100.0]	–	0/0/1	99.9 [99.6, 100.0]
b06	73	9	152	100.0	[100.0, 100.0]	–	0/0/0	96.9 [96.2, 97.6]
b03	90	30	190	87.9	[87.9, 87.9]	–	0/21/0	70.8 [69.0, 72.6]
b09	147	28	298	81.5	[98.0, 98.0]	–	36/6/0	52.8 [50.8, 54.8]
b10	167	17	358	92.5	[95.0, 95.0]	–	7/15/0	73.3 [71.5, 75.1]
b07	253	46	510	69.2	[69.2, 75.9]	–	0/100/0	56.4 [54.5, 58.4]
b13	254	53	530	35.7	[38.9, 98.7]	86.0	16/7/77	42.0 [40.0, 44.0]
b11	482	31	980	86.1	[86.3, 90.6]	–	2/75/23	64.4 [62.4, 66.3]
b04	539	66	1,102	73.2	[76.4, 89.3]	–	27/73/0	52.3 [50.3, 54.3]
b05	669	34	1,342	43.0	[43.0, 90.3]	–	0/99/1	50.4 [48.4, 52.4]
b12	999	121	2,010	36.6	[37.1, 99.9]	67.1	8/3/89	9.2 [8.1, 10.4]
b14	3,678	277	7,422	87.3	[88.1, 99.2]	88.9	44/53/3	55.3 [53.3, 57.2]
b15	6,020	418	12,114	13.7	[14.3, 100.0]	11.2	51/2/47	18.3 [16.8, 19.9]
b20	7,957	554	15,980	85.2	[85.4, 99.7]	–	30/38/32	51.8 [49.8, 53.8]
b21	8,089	554	16,244	87.3	[87.4, 99.6]	–	20/50/30	52.0 [50.0, 54.0]
b22	12,266	831	24,598*	80.7	[81.3, 98.4]	–	10/29/61	46.5 [44.5, 48.5]
b17	18,335	1,322	36,746*	8.0	[8.6, 97.3]	6.7	13/63/24	9.2 [8.1, 10.4]
b18	51,816	3,198	103,706*	3.2	[3.2, 99.6]	3.0	0/0/100	2.9 [2.2, 3.6]
b19	104,873	6,398	209,838*	1.5	[1.5, 99.4]	–	0/0/100	0.8 [0.5, 1.2]

*Sample of 2,450 faults without replacement. All shares in %. Det.: detected under the random workload. Agent: detected under the agent workload. T/U/?: formal oracle, testable (witness replayed), untestable, unknown among the sampled undecided faults.

Reproducibility. Each report records the hashes of the netlist, the instrumented netlist, every workload, and every vector file, together with the seeds and tool versions; identical inputs give identical results. Every agent exchange is stored as a prompt and response pair with the model identity and hashes, and the workloads it produced are stored as files, so that the agent results can be replayed without the LLM.